

P1687R1

Summary of the Tooling Study Group's Modules Ecosystem Technical Report Telecons

Published Proposal, 2019-08-05

Authors:

[Bryce Adelstein Lelbach](#) (NVIDIA)

[Ben Craig](#)

Source:

[GitHub](#)

Issue Tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

EWG, SG2, SG15

Table of Contents

1 Introduction

2 Summary of Meetings

3 Meeting Minutes

3.1 2019-08-02 Meeting Minutes

3.2 2019-06-21 Meeting Minutes

§ 1. Introduction

At the ISO C++ Kona 2019 meeting, the ISO C++ Committee’s Tooling Study Group, SG15, met to discuss concerns raised by various stakeholders about how modules would impact and interact with the broader C++ ecosystem (build systems, tools, other languages, etc). During that discussion, SG15 reached a consensus that the best way to prepare the C++ community for modules and ensure a smooth transition to modules over the next decade would be to prepare a [C++ Modules Ecosystem Technical Report](#). At the ISO C++ Cologne 2019 meeting, the Evolution Working Group approved starting work on this Technical Report.

The Tooling Study Group has held a series of telecons to work on the Technical Report. This document contains a summary and detailed minutes for each of the telecons.

§ 2. Summary of Meetings

Meeting	Agenda	Organizers
2019-08-02 Minutes	— D1838R0: Modules User-Facing Lexicon and File Extensions .	— <i>Chairing</i> : Bryce Adelstein Lelbach. — <i>Minute Taker</i> : Ben Craig.
2019-06-21 Minutes	— P1788: Built Module Distribution (Olga Arkhipova). — P1689: Dependency Format Specification (Ben Boeckel).	— <i>Chairing</i> : Bryce Adelstein Lelbach. — <i>Minute Taker</i> : Ben Craig.
2019-06-07 Minutes	— Plans for Pre-Cologne mailing papers. — Compiled Module Configuration (Michael Spencer). — New <code>sg15@lists.isocpp.org</code> mailing list.	— <i>Chairing</i> : Bryce Adelstein Lelbach. — <i>Minute Taker</i> : Ben Craig.

2019-05-24 Minutes	— Compiled Module Distribution (Olga Arkhipova).	— <i>Chairing</i> : Bryce Adelstein Lelbach — <i>Minute Taker</i> : Ben Craig
2019-05-10 Minutes	— P1634: Module Naming (Corentin Jabot).	— <i>Chairing</i> : Ben Craig — <i>Minute Taker</i> : Ben Craig
2019-04-26 Minutes	— Distributed Build Systems (Ben Craig, Mathias Stearn). — Clang Modules and Module Maps (Richard Smith). — New sg15@lists.isocpp.org mailing list.	— <i>Chairing</i> : Bryce Adelstein Lelbach — <i>Minute Taker</i> : Bryce Adelstein Lelbach, Ben Craig
2019-04-12 Minutes	— GCC Module Mapping: — P1184: A Module Mapper (Nathan Sidwell). — P1602: Make Me A Module (Nathan Sidwell). — New sg15@lists.isocpp.org mailing list.	— <i>Chairing</i> : Bryce Adelstein Lelbach — <i>Minute Taker</i> : Ben Craig
2019-04-05 Minutes	— P1689: Dependency Format Specification (Ben Boeckel). — Logistics of drafting the proposed C++ Ecosystem Technical Report (format, repos, etc).	— <i>Chairing</i> : Bryce Adelstein Lelbach — <i>Minute Taker</i> : Ben Craig
2019-03-22 Minutes	— Suggested outline for the proposed C++ Ecosystem Technical.	— <i>Chairing</i> : Ben Craig — <i>Minute Taker</i> : Tom Honermann
2019-03-08 Minutes	— Schedule for pre-Cologne Tooling Study Group telecons.	— <i>Chairing</i> : Bryce Adelstein Lelbach

— Scope, priorities, and goals for the proposed C++ Ecosystem Technical Report.

— *Minute Taker*: Ben Craig

§ 3. Meeting Minutes

Minutes omitted from this revision can be found in prior revisions:

— [P1687R0: 2019-03-08 through 2019-06-07.](#)

§ 3.1. 2019-08-02 Meeting Minutes

Attendance:

- Ben Craig
- Bryce Adelstein Lelbach (NVIDIA)
- Ben Boeckel (Kitware)
- Gabriel Dos Reis (Microsoft)
- Mathias Stearn
- Boris Kolpackov (build2)
- Bruno Lopes (Apple, Clang/LLVM)
- Tom Honermann (Synopsys)
- Rene Rivera (boost.build)
- Mark Zeren (VMWare)
- JF Bastien (Apple, Clang/LLVM)

Chair Notes (Bryce Adelstein Lelbach):

Agenda:

- 2019-07 Cologne summary:
 - [EWG review of P1688.](#)
 - [Friday SG15 session.](#)

- 2019-09-21 Denver CppCon meeting.
- Post-Cologne mailing (2019-08-05):
 - P1687: Tooling Telecon Minutes (Bryce Adelstein Lelbach, Ben Craig).
 - P1689: Dependency Information Format (Ben Boeckel).
 - Generalized Module Mapper (Boris Kolpackov).
- D1838R0: Modules User-Facing Lexicon and File Extensions.
 - CMI/BMI bikeshedding.
 - File extensions.

POLL: Which names can you tolerate? (Vote as many times as you like)

Name	Votes
Binary Module Interface (BMI)	10
Built Module Interface (BMI)	8
Compiled Module Interface (CMI)	6
PreCompiled Module (PCM)	4
Module Interface Metadata (MIM)	3
Module Interface Cache (MIC)	4
Importable Module Artifact (IMA)	9

POLL: Which name do you prefer?

Name	Votes
Binary Module Interface (BMI)	5
Built Module Interface (BMI)	5
Importable Module Artifact (IMA)	1

CONSENSUS: We like the acronym BMI.

Action Items:

- Publish P1687R1 with minutes from the 2019-06-21 and 2019-08-02 telecon (Bryce).
- Publish new paper with details of the 2019-09-21 Denver CppCon meeting (Bryce).
- Share D1833R0 with everybody in the Tooling Study Group (Bryce).

Minutes (Ben Craig):

- Bryce: Denver CppCon meeting. Consensus is Saturday after CppCon 2019. Sep 21. Will try to have a telecon presence. Will set up a pseudo mailing deadline. Limiting to the modules ecosystem TR.
- Bryce: Who is sending papers to the post-Cologne mailing (which is Monday)?
- Ben B: Planning on getting P1689 into the post meeting mailing.
- Boris: Planning on having a paper for generic module mapper.
- Bryce: Today we are discussing the modules lexicon paper. Want to have shared terminology in preparation for CppCon.
- Ben C: I just want consistency.
- Tom: I do like PCM because it's sort of like precompiled headers.
- Gaby: Because of that, I would go in the opposite direction. Like CMI, since it doesn't need to be stored in a binary form.
- Mathias: How do we name the act of generating the BMI / CMI? How do we need to name the act of making an object file?
- Bryce: Some impls have distance steps for making obj and CMI, some impls have one step.
- Ben B: Fortran compilers use a single step for obj and CMI.
- General disagreement on which is more efficient.
- Mathias: Been thinking of obj production as "compiling" and CMI production as extracting.
- Gaby: Maybe "metadata" instead (module interface metadata?).
- Boris: What is meta about it? what is the data?
- Gaby: Prior art, it's meta in the sense that it isn't the .obj.
- Ben B: Module interface cache? Interfaces the transient nature.
- Gaby: Would need to distinguish the term in caching systems.
- Ben B: Importable Module Artifact?
- Boris: We should keep in mind that there are a lot of existing documents that use BMI.
- If we don't have a significantly better term, we should prefer the status quo.
- If we want to be precise, then this doesn't always apply to module, but also to headers.
- Bruno: I like binary because it could mean metadata or machine code, where compiled gives impression of machine code.

POLL: Which names can you tolerate? (Vote as many times as you like)

Name	Votes
Binary Module Interface (BMI)	10
Built Module Interface (BMI)	8
Compiled Module Interface (CMI)	6
PreCompiled Module (PCM)	4
Module Interface Metadata (MIM)	3
Module Interface Cache (MIC)	4
Importable Module Artifact (IMA)	9

— Gaby: We'll need to use this in the context of builds. When we talk about headers / modules, the word module might confuse things.

POLL: Which name do you prefer?

Name	Votes
Binary Module Interface (BMI)	5
Built Module Interface (BMI)	5
Importable Module Artifact (IMA)	1

- BMI abbreviation seems to win.
- Rene: Since this might end up as a file extension we might want to keep in mind this: IMA (Mirage vector graphics EGO - Chart - Autumn), IMA (Disk image WinImage).
- Ben B: Why does it have to stand for anything? Just call it BMI.
- Bryce: Current status quo is that interface units have a different extension?
- Gaby: Only the interface has a different extension.
- Mathias: What are importable implementation units: Named partitions that are not exported Seem to behave more like interface units, because you need a BMI for them For anything that is importable. Even implementation units can be imported. Maybe don't do it at all?
- Gaby: MSVC has distinct extensions to make other tools easier. Can't change Visual Studio to not care about extension.
- Mathias: Not proposing that it doesn't have an extension, just proposing that it doesn't have a different extension.
- Gaby: Interfaces get treated a bit different from headers and implementation files. Treating an interface file as a header file isn't adequate semantically.
- Tom: This is for driving build systems?

- Gaby: This is for driving the editor and the compiler. Without the extension, you need more compiler switches to disambiguate.
- Boris: @Bryce: I don't believe GCC uses .cppm, I think you meant Clang? AFAIK, GCC/Nathan does not suggest a separate extension.
- Bryce: Overloading what .cpp means too much seems unnecessary.
- Gaby: Didn't do this lightly in 2015.
- Different extension seems useful. Suggesting .cmi.
- Ben C: Cost to existing editors that don't recognize the extension.
- Mark: Cost goes both ways, in that editors and tools can't specialize.
- Mathias: Reduced scanning on non-importable units.
- Ben B: You have to scan anything that could import as well.
- Mathias: That would be my argument that you keep all the extensions the same,.
- since all need to be scanned anyway.
- Tom: Need to support build systems without a scan step. Whole world isn't going to transition to cmake.
- Ben B: A two or three level recursive make may be able to pull off scanning. I want to make a paper that says how our generated makefiles work.
- Bryce: On produced files, hopefully the implementations will coalesce on an extension. MSVC uses .ifc. Want to avoid .o or .obj mismatch, but this isn't as high of a priority. GCC doesn't seem to support telling it which extension to use.
- Gaby: Implementation unit, I see .cpp and .mxx. GCC allows .cc and .cxx.
- Bryce: Not currently exhaustive. .cpp is a placeholder for all the existing extensions.
- Ben B: GCC uses gcm, gcmu, or gcms depending on the type.
- Boris: I think Nathan changed that.
- Ben B: Default module mapper uses CWD and makes up names, if you give it a module map, it uses that.
- Boris: @Bryce: with GCC you can write the BMI to any file (any name/extension, etc) using the module mapper.
- Bikeshedding over "Dependency Metadata" / "Dependency Information".
- Mathias: Rather not use implicit vs. explicit for this distinction. Explicit sounds like users are managing it.
- Maybe compiler managed, build system managed, user managed.

- Some discussion over merging Mathias's categories.
- Gaby: Please don't use the terms implicit and explicit terms for "Implicit Modules" and "Explicit Modules".
- Bryce: This should probably be considered anti-terminology.
- Ben C: Need different words for textual `#include` , header unit `#include` , and module import `foo`;

§ 3.2. 2019-06-21 Meeting Minutes

Attendance:

- Ben Craig
- Bryce Adelstein Lelbach (NVIDIA)
- Ben Boeckel (Kitware)
- David Blaikie (Google, Clang/LLVM)
- Rene Rivera (Boost.Build)
- JF Bastien (Apple, Clang/LLVM)
- Lukasz Mendakiewicz (Microsoft, MSVC)
- Nathan Sidwell (Facebook, GCC)
- Tom Honermann (Synopsys)
- Olga Arkhipova (Microsoft)
- Steve Downey
- Mark Zeren (VMWare)
- Michael Spencer (Apple, Clang/LLVM)
- Matthew Woelke (Kitware)
- Richard Smith (Google, Clang/LLVM)

Chair Notes (Bryce Adelstein Lelbach):

- Agenda:
- New mailing list on lists.isocpp.org.
- Cologne prep
 - Who actually sent papers?

- Review Ben’s Dependency Metadata Paper (P1689).
- Review Olga’s BMI Distribution Paper (P1788).
- Review Richard’s Packaging Paper (P1767).
- Next (last) Pre-Cologne call is July 5th (reschedule?).
- Pre-Cologne Papers (Send Drafts to sg15@lists.isocpp.org by 2019-06-14):
- P1687: Summary and Minutes of Pre-Cologne SG15 Discussions (Bryce, Ben C).
- P1688: Ecosystem Technical Report Outline (Bryce).
- P1689: Dependency Metadata (Ben B).
- P1788: Compiled Module Reuse (Olga, Lukasz).
- Module Naming (Corentin Jabot).
- Modules Hello World (Gaby).
- P1767: Modules and Packaging (Richard).
- RFE: (Paper or info on) Implicit Modules (Michael).
- Michael gave a talk at LLVM Euro about this.

Minutes (Ben Craig):

Ben Boeckel presents [P1689](#).

- Bryce: Did the unicode stuff come up last time, and did you talk to the unicode group?
- Ben B: It was discussed on slack.
- Tom: Worth a small presentation to SG16, looks like a good approach to the problem.
- Bryce: Unfortunate that this paper needs to deal with it, as it seems like a JSON problem.
- Tom: Beyond the scope of just json. It’s something that has come up in other contexts too.
- Bryce: working directory and chroot stuff is new, right?
- Ben B: yes.
- Ben B: future compile and future link are new. Want to be able to support #pragma comment for auto linking.
- Zeren: How important is windows auto linking? We rip it out of boost.
- Ben B: It came up in slack, I’m not super attached to it.
- Olga: I don’t know if it’s needed.
- Nathan: Could be something in vendor extensions.

- Ben B: vendor extensions are supposed to be ignorable.
- Ben B: May be able to leave auto linking out of the first version.
- Ben C: Maybe std::embed like cases?
- Ben B: I don't think a C++ file mentions linker embedded things.
- Ben C: Agreed. Withdrawn.
- Tom: Seems like a good vendor extension to me.
- Bryce: Doesn't seem to come up with our weird heterogenous context.
- Rene: shouldn't the vendor extensions say whether it is required or not?
- Ben B: For vendor extensions, if it is required and necessary for a correct build, then we wouldn't know how to turn that into something to make ninja or make work.
- Bryce: Similar to C++ attributes.
- Tom: These are intended to be transient files. Not meant to be shipped out. May even be piped and not stored on file system.
- Ben B: yes.
- Bryce: Never seen anyone store off a makefile database to be used later.
- Rene: I have.
- Rene: The unreal build system will serialize out some intermediate files and use the same one repeatedly.
- Bryce: Just trying to ensure that the intent is that this file doesn't go into source control.
- Ben B: tup is a build tool where each rule gets a chroot, they will need to use the working directory in interesting ways.

Olga and Lukasz present [P1788](#).

- Bryce: Have LLVM people had a chance to see the paper?
- Spencer: Yes, mostly makes sense. Nothing in here surprising or hard?
- Tom: Looking at the recommendation to library vendors, does LLVM agree that it is ok to ship BMIs as an optimization?
- Spencer: Yes, that's fine. BMI as an optimization is fine.
- Tom: Will you ignore them if they are inappropriate?
- Spencer: You will get an error if you give us a bad one explicitly, we will rebuild if it is implicit. We can't easily rebuild for explicit, because we don't know how to build it correctly.

- Tom: You could ask the BMI how to build it.
- Ben C: But that information is only how to build that BMI, now how to build it correctly.
- Richard S: Including env and working directory could be trouble. We need bit for bit build reproducibility, even when different working directories are used to build on different machines.
- Tom: May also need to include which important env variables are unset, so that those can be cleared.
- David: Distributing prebuilt modules as an optimization... sounded like a bit of a disconnect. In explicit modules world, the scanning or oracle system would need to be able to see if those BMIs are suitable or not. If the prebuilt modules are good, the scan would report that prebuilt module.
- Nathan: I've started putted in human readable section of BMI things like working directory. It doesn't effect the CRCs, and those sections could be stripped to possibly get bit for bit reproducibility.
- Bryce: Is this something that GCC can get on board with?
- Nathan: Putting information to rebuild from source may only want to be done explicitly, because it might be expensive, and it might be a secret.
- Tom: Would you be amenable to adding an ENV variable to get that information into a BMI.
- Tom: Intent is that all of the source needs to get into a BMI. Particularly because of header files that went into it.
- Nathan: Already hit issues where header files of library on producing machine are different from header files on consuming machine.
- Tom: We really need all the source.
- Olga: Could include the header file paths.
- Tom: But those files may not be available on the consuming machine.
- Lukasz: Debugger has some hashes. May want to be able to ignore whitespace changes.
- Olga: Directory structure of location where BMI is built may be different than directory structure where BMI is used?
- Olga: If you can't find those files using the command line, then full paths won't help. Are you asking for the BMI to include the source itself?
- Nathan: compiler vendors may want that to be able to deal with bug reports.
- Matthew W: Should be a way for linux distros to point to them instead of vendoring them.
- Nathan: My experience is with multiple red hat based systems that are still different enough to cause problems.

- Matthew W: Wanted to go back and say something about whether you can use a prebuilt BMI. I would punt on the problem. Wouldn't have the scanning tool fix it. Let the compiler have something like ccache sitting in front of it that looks in a cache to see if the compile line is compatible. If it's not there then it can be built at that time.
- Steve D: Anyone shipping a BMI has to be prepared to have consumers rebuild the BMI. It's an important optimization, but the BMI can't be trusted... you need to be ready to scrap it and rebuild it.
- Bryce: Main point of contentions are the "Other build options" bullet point and "module source" bullet point.
- Olga: Those things can be ignored by users, but I need to think about it.
- Tom: There are certainly copyright concerns.